LAPORAN PRAKTIKUM
ALGORITMA & STRUKTUR DATA
MODUL 2

STACK DAN QUEUE

Oleh:

Muhammad Dani Maulana **NIM. 2510817110012**

PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
APRIL 2026

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM ALGORITMA & STRUKTUR DATA
MODUL 2

Laporan Praktikum Algoritma & Struktur Data Modul 2: Stack dan Queue ini disusun sebagai syarat lulus mata kuliah Praktikum Algoritma & Struktur Data. Laporan Prakitkum ini dikerjakan oleh:

Nama Praktikan : Muhammad Dani Maulana
NIM : 2510817110012

Menyetujui, Asisten Praktikum	Mengetahui, Dosen Penanggung Jawab Praktikum
Muhammad Fauzan Ahsani NIM. 2310817310009	Ir. Muhamamd Alkaff, S,Kom, M.Kom, Ph.D NIP. 198606132015041001

DAFTAR ISI

LEMBAR PENGESAHAN i
DAFTAR ISI ii
DAFTAR GAMBAR iii
DAFTAR TABEL iv
SOAL 1 1
A. Pembahasan 1
SOAL 2: Si Palui dan Mesin Hitung Aneh 3
A. Source Code 4
B. Output Program 7
C. Pembahasan 7
SOAL 3 14
A. Pembahasan 14
SOAL 4: Si Palui dan Deretan Angka 15
A. Source Code 16
B. Output Program 18
C. Pembahasan 19
TAUTAN GIT 24

DAFTAR GAMBAR

Gambar 1. Screenshot hasil jawaban Soal 1A 7
Gambar 2. Screenshot hasil jawaban Soal 1B 7
Gambar 3. Screenshot hasil jawaban Soal 1C 7
Gambar 4. Screenshot hasil jawaban Soal 2A 18
Gambar 5. Screenshot hasil jawaban Soal 2B 19
Gambar 6. Screenshot hasil jawaban Soal 2C 19

DAFTAR TABEL

Table 1. Source Code prob1.cpp 4
Table 2. Source Code stack.cpp 5
Table 3. Source Code stack.h 6
Table 4. Source Code prob1.cpp 16
Table 5. Source Code queue.cpp 16
Table 6. Source Code queue.h 18

SOAL 1

Pada file stack.h yang diberikan, implementasikan seluruh fungsi yang dideklarasikan ke dalam file stack.cpp. Untuk menangani kondisi tidak valid (invalid case), gunakan mekanisme exception handling dengan throw.

Jelaskan alur algoritma dari setiap fungsi yang diimplementasikan, serta berikan justifikasi atau pembuktian singkat mengenai kebenaran algoritma tersebut. Selain itu, lakukan analisis kompleksitas waktu dan ruang untuk setiap fungsi yang ada. (Gunakan Big-O Notation).

A. Pembahasan

Fungsi init bertujuan untuk menginisialisasi stack ke kondisi awal, yaitu kosong. Algoritmanya sangat sederhana, hanya dengan mengatur pointer top menjadi nullptr. Hal ini menandakan bahwa belum ada elemen dalam stack. Kebenaran algoritma ini didasarkan pada invariant bahwa stack kosong selalu direpresentasikan dengan top == nullptr, sehingga fungsi lain seperti isEmpty dapat bekerja dengan konsisten. Kompleksitas waktu dan ruang dari fungsi ini adalah O(1) karena hanya melakukan satu operasi sederhana tanpa alokasi tambahan.

Fungsi isEmpty digunakan untuk mengecek apakah stack dalam kondisi kosong. Algoritmanya hanya membandingkan apakah top bernilai nullptr. Jika iya, maka stack kosong. jika tidak, berarti terdapat elemen di dalamnya. Kebenaran fungsi ini mengikuti definisi struktur stack yang menggunakan pointer sebagai penanda elemen teratas. Karena hanya melakukan satu perbandingan, kompleksitas waktu fungsi ini adalah O(1) dan tidak membutuhkan ruang tambahan sehingga kompleksitas ruangnya juga O(1).

Fungsi isFull bertujuan untuk mengecek apakah stack sudah penuh. Karena stack diimplementasikan menggunakan array dengan ukuran tetap, kondisi penuh terjadi ketika pointer top sudah menunjuk ke elemen terakhir dalam array. Algoritmanya membandingkan top dengan alamat elemen terakhir data[MAX-1]. Jika sama, berarti stack tidak dapat menerima elemen baru. Kebenaran algoritma ini berasal dari sifat array statis yang memiliki batas kapasitas tetap. Kompleksitas waktu dan ruang fungsi ini adalah O(1) karena hanya melakukan satu operasi perbandingan.

Fungsi push digunakan untuk menambahkan elemen baru ke dalam stack. Algoritma dimulai dengan mengecek apakah stack penuh menggunakan isFull. jika penuh, maka akan melempar exception. Jika tidak penuh, maka terdapat dua kondisi:
jika stack kosong, top akan diarahkan ke elemen pertama array
jika tidak kosong, pointer top akan dinaikkan satu posisi.
Setelah itu, nilai baru disimpan pada posisi top. Kebenaran algoritma ini didasarkan pada prinsip LIFO (Last In First Out), di mana elemen terakhir yang dimasukkan selalu berada di posisi teratas. Kompleksitas waktu fungsi ini adalah O(1) karena hanya melibatkan operasi pointer dan assignment, serta kompleksitas ruangnya O(1).

Fungsi pop berfungsi untuk menghapus dan mengembalikan elemen teratas dari stack. Algoritmanya dimulai dengan mengecek apakah stack kosong; jika kosong, maka akan melempar exception. Jika tidak kosong, nilai pada top disimpan terlebih dahulu. Kemudian, jika elemen tersebut adalah satu-satunya elemen, maka top diatur kembali menjadi nullptr. Jika masih ada elemen lain, pointer top akan diturunkan satu posisi. Nilai yang disimpan kemudian dikembalikan. Kebenaran algoritma ini terletak pada penerapan prinsip LIFO, di mana elemen terakhir yang masuk adalah yang pertama keluar. Kompleksitas waktu fungsi ini adalah O(1) dan kompleksitas ruangnya juga O(1).

Fungsi peek digunakan untuk melihat elemen teratas stack tanpa menghapusnya. Algoritmanya adalah dengan mengecek apakah stack kosong; jika kosong, akan melempar exception. Jika tidak, fungsi langsung mengembalikan nilai yang ditunjuk oleh top. Kebenaran algoritma ini jelas karena tidak mengubah struktur stack dan hanya membaca nilai teratas. Kompleksitas waktu dan ruang fungsi ini adalah O(1) karena hanya melakukan satu operasi akses.

Fungsi HasilOperasi digunakan untuk menghitung hasil dari ekspresi dalam bentuk Reverse Polish Notation (RPN). Algoritmanya melakukan iterasi terhadap setiap elemen input. Jika elemen berupa operator, maka dua elemen teratas di-pop dari stack, kemudian dilakukan operasi sesuai operator tersebut, dan hasilnya di-push kembali ke stack. Jika elemen berupa angka, maka dikonversi ke integer menggunakan stoi, divalidasi agar berada dalam rentang tertentu, lalu dimasukkan ke dalam stack. Setelah seluruh input diproses, hasil akhir diambil dari stack. Kebenaran algoritma ini didasarkan pada metode standar evaluasi postfix yang memanfaatkan struktur stack untuk memastikan urutan operasi tetap benar.

Kompleksitas waktu fungsi ini adalah O(n) karena memproses setiap elemen sekali, sedangkan kompleksitas ruangnya adalah O(n) karena stack dapat menyimpan hingga n elemen.

SOAL 2: Si Palui dan Mesin Hitung Aneh

Si Palui menemukan sebuah mesin hitung tua di gudang. Mesin tersebut memiliki cara kerja yang tidak biasa. Alih-alih menggunakan format perhitungan seperti yang biasa kita kenal, mesin ini membaca input berupa deretan simbol yang dipisahkan oleh spasi.

Setiap simbol bisa berupa:

- sebuah bilangan bulat, atau
- sebuah operator (+, -, *, /)

Cara kerja mesin adalah sebagai berikut:

- Mesin membaca simbol dari kiri ke kanan.
- Jika simbol adalah bilangan, maka bilangan tersebut disimpan ke dalam memori mesin.
- Jika simbol adalah operator, maka mesin akan mengambil dua nilai yang paling baru dimasukkan ke dalam memori yang disimpan, melakukan operasi sesuai operator tersebut, lalu menyimpan kembali hasilnya ke dalam memori.

Setelah seluruh simbol diproses, akan tersisa satu nilai di dalam memori. Nilai inilah yang ditampilkan sebagai hasil akhir. Sebagai asisten Si Palui, tugas Anda adalah membantu menghitung hasil dari mesin tersebut.

Batasan

- $1 \leq n \leq 100$
- $-1000 \leq Ai \leq 1000$

Keterangan:

- n adalah jumlah simbol dalam ekspresi
- Ai adalah nilai bilangan pada input
- Simbol yang valid hanya:
  → bilangan bulat
  → operator: +, -, *, /

Format Input

```
n
A1 A2 ... An
```

Keterangan:

- Baris pertama n menyatakan jumlah atau panjang simbol
- Baris kedua berisi simbol A sebanyak n buah

Format Output

Sebuah bilangan hasil dari kalkulator Si Palui.

Contoh Kasus

Input	Output
3 2 3 +	5
9 5 1 2 + 4 * + 3 -	14
5 10 2 / 3 *	15

A. Source Code

Table 1. Source Code prob1.cpp

```
#include <iostream>
#include <string>
#include "stack.h"
using namespace std;

int main(){
    Stack s;
    init(&s);
    string inputData[MAX];

    int n;
    if (!(cin >> n)) return 0;

    for (int i = 0; i < n && i < MAX; i++) {
        cin >> inputData[i];
    }
    int hasil = HasilOperasi(&s, n, inputData);
    cout << hasil << endl;
    return 0;
}
```

Table 2. Source Code stack.cpp

```
#include "stack.h"
#include <iostream>
#include <string>
using namespace std;

void init(Stack* s){
    s->top = nullptr;
}

bool isEmpty(const Stack* s){
    return s->top == nullptr;
}

bool isFull(const Stack* s){
    return s->top == &(s->data[MAX - 1]);
}

void push(Stack* s, int value){
    if (isFull(s)) throw "Stack penuh";

    if (isEmpty(s)) {
        s->top = s->data;
    } else {
        s->top++;
    }
    *s->top = value;
}

int pop(Stack* s){
    if (isEmpty(s)) throw "Stack kosong";
    int value = *s->top;
    if (s->top == s->data) {
        s->top = nullptr;
    } else {
        s->top--;
    }
    return value;
}

int peek(const Stack* s){
    if (isEmpty(s)) throw "Stack kosong";
    return *s->top;
}

```

```
int HasilOperasi(Stack* s, int n, string input[]){
    for (int i = 0; i < n; i++) {
        string val = input[i];

        if(val == "+" || val == "-" || val == "*" || val
== "/"){
            int bil2 = pop(s);
            int bil1 = pop(s);
            int hasil;

            if(val == "+") hasil = bil1 + bil2;
            else if(val == "-") hasil = bil1 - bil2;
            else if(val == "*") hasil = bil1 * bil2;
            else if(val == "/") {
                if(bil2 == 0) hasil = 0;
                else hasil = bil1 / bil2;
            }

            push(s, hasil);
        } else {
            int bil = stoi(val);
            if (bil < -1000 || bil > 1000) {
                throw "Bilangan diluar jangkauan";
            }
            push(s, bil);
        }
    }

    return isEmpty(s) ? 0: pop(s);
}
```

Table 3. Source Code stack.h

```
#ifndef STACK_H
#define STACK_H

#define MAX 100
#include <string>

struct Stack {
    int data[MAX];
    int* top;
};

void init(Stack* s);
bool isEmpty(const Stack* s);
bool isFull(const Stack* s);
```

```
void push(Stack* s, int value);
int pop(Stack* s);
int peek(const Stack* s);
int HasilOperasi(Stack* s, int n, std::string input[]);

#endif
```

B. Output Program

```
bin\WindowsDebugLauncher.exe' '--stdin=Microsoft-MIEngine-In-lwvvxvem.zpr' '--stdout=Microsoft-MIEngine-Out-l
e-Pid-cldtjcvi.55w' '--dbgExe=C:\MinGW\bin\gdb.exe' '--interpreter=mi'
Masukkan Jumlah Bilangan:
3
2 3 +

5
```

Gambar 1. Screenshot hasil jawaban Soal 1A

```
PS D:\FILE DANI\KULIAH\SEMESTER 2\ALGORITMA P2\PRAKTIKUM\MODUL 2\module-2-stack-and-queu
bin\WindowsDebugLauncher.exe' '--stdin=Microsoft-MIEngine-In-xzgbq3xt.qbc' '--stdout=Mic
e-Pid-nzfz2v4a.vu1' '--dbgExe=C:\MinGW\bin\gdb.exe' '--interpreter=mi'
Masukkan Jumlah Bilangan:
9
5 1 2 + 4 * + 3 -

14
```

Gambar 2. Screenshot hasil jawaban Soal 1B

```
PS D:\FILE DANI\KULIAH\SEMESTER 2\ALGORITMA P2\PRAKTIKUM\MODUL 2\mod
bin\WindowsDebugLauncher.exe' '--stdin=Microsoft-MIEngine-In-41j0uod
e-Pid-bfckvxzm.p1k' '--dbgExe=C:\MinGW\bin\gdb.exe' '--interpreter=m
Masukkan Jumlah Bilangan:
5
10 2 / 3 *

15
```

Gambar 3. Screenshot hasil jawaban Soal 1C

C. Pembahasan

Prob1.cpp:

- `#include <iostream>`

  Digunakan untuk input dan output (cin, cout).

- `#include <string>`

  Digunakan agar bisa memakai tipe data string.
- `#include "stack.h"`

  Mengimpor implementasi stack dan fungsi yang dibutuhkan (seperti init).
- `using namespace std;`

  Supaya tidak perlu menulis std:: setiap menggunakan cin, cout, string, dll.
- `int main()`

  Fungsi utama tempat program mulai dieksekusi.
- `Stack s;`

  Membuat variabel stack bernama s.
- `init(&s);`

  Menginisialisasi stack agar siap digunakan (biasanya mengatur top = -1).
- `string inputData[MAX];`

  Menyediakan array untuk menyimpan input dalam bentuk string.
- `int n;`

  Variabel untuk menyimpan jumlah data yang akan dimasukkan.
- `if (!(cin >> n)) return 0;`

  Membaca input n, jika gagal maka program langsung berhenti.
- `for (int i = 0; i < n && i < MAX; i++)`

  Perulangan untuk mengambil input sebanyak n data, dengan batas maksimum MAX.
- `cin >> inputData[i]`

  Menyimpan setiap input ke dalam array inputData.
- `int hasil = HasilOperasi(&s, n, inputData);`

  Memanggil fungsi untuk memproses data (kemungkinan evaluasi ekspresi) dan menyimpan hasilnya.
- `cout << hasil << endl;`

  Menampilkan hasil perhitungan.
- `return 0;`

  Mengakhiri program dengan status sukses.

Stack.cpp:

- `#include "stack.h"`

  Menghubungkan file ini dengan deklarasi struct dan fungsi stack.
- `#include <iostream>`

  Digunakan untuk input/output (misalnya cout).
- `#include <string>`

  Digunakan untuk tipe data string.
- `using namespace std;`

  Supaya bisa pakai cout, string, dll tanpa std::.
- `void init(Stack* s){`

  Fungsi untuk inisialisasi stack.
- `s->top = nullptr;`

  Menandakan stack masih kosong.
- `bool isEmpty(const Stack* s){`

  Fungsi untuk mengecek apakah stack kosong.
- `return s->top == nullptr;`

  Jika top null, berarti stack kosong.
- `bool isFull(const Stack* s){`

  Fungsi untuk mengecek apakah stack penuh.
- `return s->top == &(s->data[MAX - 1]);`

  Jika top sudah di elemen terakhir array, berarti penuh.
- `void push(Stack* s, int value){`

  Fungsi untuk menambah elemen ke stack.
- `if (isFull(s)) throw "Stack penuh";`

  Jika penuh, tampilkan error.
- `if (isEmpty(s)) {`

  Jika stack kosong.
- `s->top = s->data;`

  top diarahkan ke elemen pertama array.
- `else {`

  Jika tidak kosong.

- `s->top++;`

  top naik ke posisi berikutnya.
- `*s->top = value;`

  Simpan nilai ke posisi top.
- `int pop(Stack* s){`

  Fungsi untuk mengambil dan menghapus elemen teratas.
- `if (isEmpty(s)) throw "Stack kosong";`

  Jika kosong, tampilkan error.
- `int value = *s->top;`

  Ambil nilai di posisi top.
- `if (s->top == s->data) {`

  Jika hanya ada satu elemen.
- `s->top = nullptr;`

  Reset jadi kosong.
- `else {`

  Jika lebih dari satu elemen.
- `s->top--;`

  Turunkan posisi top.
- `return value;`

  Mengembalikan nilai yang di-pop.
- `int peek(const Stack* s){`

  Fungsi untuk melihat nilai teratas tanpa menghapus.
- `if (isEmpty(s)) throw "Stack kosong";`

  Jika kosong, tampilkan error.
- `return *s->top;`

  Mengembalikan nilai di top.
- `int HasilOperasi(Stack* s, int n, string input[]){`

  Fungsi untuk menghitung operasi (format postfix/RPN).
- `for (int i = 0; i < n; i++) {`

  Loop untuk membaca semua input.
- `string val = input[i];`

Ambil nilai input ke-i.

- `if(val == "+" || val == "-" || val == "*" || val == "/"){`

  Jika input adalah operator.

- `int bil2 = pop(s);`

  Ambil operand kedua.

- `int bil1 = pop(s);`

  Ambil operand pertama.

- `int hasil;`

  Variabel untuk menyimpan hasil.

- `if(val == "+") hasil = bil1 + bil2;`

  Penjumlahan.

- `else if(val == "-") hasil = bil1 - bil2;`

  Pengurangan.

- `else if(val == "*") hasil = bil1 * bil2;`

  Perkalian.

- `else if(val == "/") {`

  Pembagian.

- `if(bil2 == 0) hasil = 0;`

  Jika pembagi 0, hasil dibuat 0.

- `else hasil = bil1 / bil2;`

  Jika tidak, lakukan pembagian.

- `push(s, hasil);`

  Simpan hasil kembali ke stack.

- `else {`

  Jika bukan operator (berarti angka).

- `int bil = stoi(val);`

  Ubah string jadi integer.

- `if (bil < -1000 || bil > 1000) {`

  Cek batas nilai.

- `throw "Bilangan diluar jangkauan";`

  Jika di luar batas, error.

- `push(s, bil);`

  Masukkan angka ke stack.
- `return isEmpty(s) ? 0: pop(s);`

  Jika stack kosong, return 0; jika tidak, ambil hasil akhir.

stack.h:

- `#ifndef STACK_H`

  Mengecek apakah file header ini belum pernah di-include.
- `#define STACK_H`

  Menandai bahwa file ini sudah digunakan agar tidak terjadi duplikasi.
- `#define MAX 100`

  Menentukan kapasitas maksimum stack adalah 100 elemen.
- `#include <string>`

  Digunakan untuk tipe data string.
- `struct Stack {`

  Mendefinisikan struktur data bernama Stack.
- `int data[MAX];`

  Array untuk menyimpan elemen stack.
- `int* top;`

  Pointer yang menunjuk ke elemen paling atas.
- `void init(Stack* s);`

  Deklarasi fungsi untuk menginisialisasi stack.
- `bool isEmpty(const Stack* s);`

  Deklarasi fungsi untuk mengecek apakah stack kosong.
- `bool isFull(const Stack* s);`

  Deklarasi fungsi untuk mengecek apakah stack penuh.
- `void push(Stack* s, int value);`

  Deklarasi fungsi untuk menambahkan elemen ke stack.
- `int pop(Stack* s);`

  Deklarasi fungsi untuk mengambil dan menghapus elemen teratas.
- `int peek(const Stack* s);`

Deklarasi fungsi untuk melihat elemen teratas tanpa menghapus.

- `int HasilOperasi(Stack* s, int n, std::string input[]);`

  Deklarasi fungsi untuk menghitung operasi postfix (RPN).

- `#endif`

  Penutup dari header guard.

SOAL 3

Pada file queue.h yang diberikan, implementasikan seluruh fungsi yang dideklarasikan ke dalam file queue.cpp. Untuk menangani kondisi tidak valid (invalid case), gunakan mekanisme exception handling dengan throw.

Jelaskan alur algoritma dari setiap fungsi yang diimplementasikan, serta berikan justifikasi atau pembuktian singkat mengenai kebenaran algoritma tersebut. Selain itu, lakukan analisis kompleksitas waktu dan ruang untuk setiap fungsi yang ada. (Gunakan Big-O Notation).

A. Pembahasan

`init(Queue* q)` Fungsi ini menginisialisasi queue dengan mengatur pointer `front` dan `rear` ke `nullptr`, yang menandakan queue kosong. Algoritmanya sederhana dan selalu benar karena kondisi kosong didefinisikan dengan `front == nullptr`. Kompleksitas waktunya O(1) dan ruangnya O(1).

`isEmpty(const Queue* q)` Fungsi ini mengecek apakah queue kosong dengan membandingkan `front` terhadap `nullptr`. Jika ya, maka queue kosong. Ini benar karena tidak ada elemen yang ditunjuk. Kompleksitas waktu dan ruang adalah O(1).

`isFull(const Queue* q)` Fungsi ini menentukan apakah queue penuh dengan konsep circular queue, yaitu saat posisi berikutnya dari `rear` sama dengan `front`. Ini memastikan tidak terjadi overflow. Algoritma ini benar karena memanfaatkan sifat melingkar array. Kompleksitas waktu dan ruang adalah O(1).

`enqueue(Queue* q, int value)` Fungsi ini menambahkan elemen ke belakang queue. Jika penuh, akan melempar exception. Jika kosong, `front` dan `rear` diarahkan ke elemen pertama; jika tidak, `rear` digeser secara melingkar lalu nilai dimasukkan. Ini menjaga urutan FIFO. Kompleksitas waktu O(1) dan ruang O(1).

`dequeue(Queue* q)` Fungsi ini menghapus elemen dari depan queue. Jika kosong, akan melempar exception. Jika hanya satu elemen, queue dikosongkan kembali; jika lebih, `front` digeser maju. Ini benar karena selalu menghapus elemen pertama (FIFO). Kompleksitas waktu O(1) dan ruang O(1).

`front(const Queue* q)` Fungsi ini mengembalikan nilai elemen paling depan tanpa menghapusnya. Jika queue kosong, akan melempar exception. Ini benar karena `front` selalu menunjuk elemen pertama. Kompleksitas waktu dan ruang adalah O(1).

`back(const Queue* q)` Fungsi ini mengembalikan nilai elemen paling belakang. Jika queue kosong, akan melempar exception. Ini benar karena `rear` selalu menunjuk elemen terakhir yang dimasukkan. Kompleksitas waktu dan ruang adalah O(1).

`hasilQueue(int n, int k, int inputData[])` Fungsi ini menghitung jumlah setiap subarray berukuran `k` menggunakan queue. Untuk setiap posisi, queue diisi `k` elemen lalu dijumlahkan dengan proses dequeue. Algoritma ini benar karena setiap window dihitung secara lengkap. Kompleksitas waktunya $O(n \cdot k)$ dan ruangnya $O(k)$.

SOAL 4: Si Palui dan Deretan Angka

Si Palui memiliki sebuah deretan angka yang tersusun rapi. Ia tertarik untuk mengamati sebagian angka secara berurutan dengan jumlah tertentu.

Ia memilih sejumlah k angka yang bersebelahan, lalu menghitung jumlahnya. Setelah itu, ia menggeser pilihannya satu langkah ke kanan dan kembali menghitung jumlah dari angka-angka yang terpilih.

Proses ini dilakukan terus hingga tidak memungkinkan lagi untuk memilih k angka berurutan.

Sebagai asistennya, bantulah Si Palui untuk menentukan hasil perhitungan tersebut.

Batasan

- $1 \leq n \leq 100$
- $-1 \leq k \leq n$
- $-1000 \leq Ai \leq 1000$

Keterangan:

- n: jumlah elemen
- k: ukuran jendela
- Ai: elemen array

Format Input

n k

A1 A2 ... An

Keterangan:

- Baris pertama n menyatakan jumlah atau panjang array. Lalu k besar atau jumlah elemen dalam satu kelompok ayng dibentuk
- Baris kedua berisi simbol A sebanyak n buah Format Output Cetak hasil tiap masing-masing kelompok dalam satu baris dipisahkan dengan spasi

Contoh Kasus

Input	Output
5 3 1 2 3 4 5	6 9 12
5 5 1 2 3 4 5	15
6 2 4 -1 2 -3 5 1	3 1 -1 2 6

A. Source Code

Table 4. Source Code prob2.cpp

```
#include <iostream>
#include "queue.h"

using namespace std;

int main() {
    int n, k;
    if (!(cin >> n >> k)) return 0;
    int inputData[MAX];
    for (int i = 0; i < n && i < MAX; i++) {
        cin >> inputData[i];
    }
    hasilQueue(n, k, inputData);
    return 0;
}
```

Table 5. Source Code queue.cpp

```
#include "queue.h"
#include <iostream>
using namespace std;

void init(Queue* q) {
    q->front = nullptr;
    q->rear = nullptr;
}

bool isEmpty(const Queue* q) {
    return q->front == nullptr;
}

bool isFull(const Queue* q) {
```

```
    if (isEmpty(q)) return false;
    int nextRearIdx = (q->rear - q->data + 1) % MAX;
    return (q->data + nextRearIdx) == q->front;
}

void enqueue(Queue* q, int value) {
    if (isFull(q)) throw "Queue penuh";
    if (isEmpty(q)) {
        q->front = q->data;
        q->rear = q->data;
    } else {
        int nextRearIdx = (q->rear - q->data + 1) % MAX;
        q->rear = q->data + nextRearIdx;
    }
    *(q->rear) = value;
}

void dequeue(Queue* q) {
    if (isEmpty(q)) throw "Queue kosong";
    if (q->front == q->rear) {
        q->front = nullptr;
        q->rear = nullptr;
    } else {
         int nextFrontIdx = (q->front - q->data + 1) %
MAX;
        q->front = q->data + nextFrontIdx;
    }
}

int front(const Queue* q) {
    if (isEmpty(q)) throw "Queue kosong";
    return *(q->front);
}

int back(const Queue* q) {
    if (isEmpty(q)) throw "Queue kosong";
    return *(q->rear);
}

void hasilQueue(int n, int k, int inputData[]){
    Queue q;
    for (int i = 0; i <= n - k; i++) {
        init(&q);
        for (int j = i; j < i + k; j++) {
            enqueue(&q, inputData[j]);
        }

```

```
        long long currentSum = 0;
        while (!isEmpty(&q)) {
            currentSum += front(&q);
            dequeue(&q);
        }

        cout << currentSum << (i == n - k ? "" : " ");
    }
    cout << endl;
}
```

Table 6. Source Code queue.h

```
#ifndef QUEUE_H
#define QUEUE_H

#define MAX 100

struct Queue {
    int data[MAX];
    int* front;
    int* rear;
};

void init(Queue* q);
bool isEmpty(const Queue* q);
bool isFull(const Queue* q);
void enqueue(Queue* q, int value);
void dequeue(Queue* q);
int front(const Queue* q);
int back(const Queue* q);
void hasilQueue(int n, int k, int inputData[]);

#endif
```

B. Output Program

```
win32-x64\debugAdapters\bin\WindowsDebugLauncher.exe' '--stdin=Mic
ine-Error-wpxh4isu.ot5' '--pid=Microsoft-MIEngine-Pid-uzr543jp.znc
Masukkan n dan k:
5 3
1 2 3 4 5
6 9 12
```

Gambar 4. Screenshot hasil jawaban Soal 2A

```
PS D:\FILE DANI\KULIAH\SEMESTER 2\ALGORITMA P2\PRAKTIKUM\MOD
win32-x64\debugAdapters\bin\WindowsDebugLauncher.exe' '--st
ine-Error-lr23warg.umq' '--pid=Microsoft-MIEngine-Pid-uwklf
Masukkan n dan k:
5 5
1 2 3 4 5
15
```

Gambar 5. Screenshot hasil jawaban Soal 2B

```
PS D:\FILE DANI\KULIAH\SEMESTER 2\ALGORITMA P2\PRAKTIKUM\MODUL 2\modu
win32-x64\debugAdapters\bin\WindowsDebugLauncher.exe' '--stdin=Micros
ine-Error-5erozwqr.er2' '--pid=Microsoft-MIEngine-Pid-rdpw4rly.wkz' '
Masukkan n dan k:
6 2
4 -1 2 -3 5 1
3 1 -1 2 6
```

Gambar 6. Screenshot hasil jawaban Soal 2C

C. Pembahasan

Prob2.cpp:

- `#include <iostream>`

  Digunakan untuk input dan output (cin, cout).
- `#include "queue.cpp"`

  Mengimpor implementasi queue dan fungsi hasilQueue.
- `using namespace std;`

  Agar tidak perlu menulis std::.
- `int main() {`

  Fungsi utama tempat program dijalankan.
- `int n, k;`

  Variabel:
  - n = jumlah data
  - k = parameter tambahan (biasanya untuk logika tertentu di queue, misalnya window/sliding atau operasi tertentu)
- `if (!(cin >> n >> k)) return 0;`

Membaca input n dan k, jika gagal maka program berhenti.

- `int inputData[MAX];`

  Array untuk menyimpan data yang dimasukkan user.
- `for (int i = 0; i < n && i < MAX; i++)`

  Perulangan untuk membaca data sebanyak n, dengan batas maksimum MAX.
- `cin >> inputData[i];`

  Menyimpan setiap input ke array.
- `hasilQueue(n, k, inputData);`

  Memanggil fungsi utama untuk memproses data menggunakan queue sesuai logika soal.
- `return 0;`

  Mengakhiri program dengan status sukses.

queue.cpp:

- `init(Queue* q)`

  Fungsi ini digunakan untuk menginisialisasi queue. front dan rear di-set ke nullptr yang artinya queue masih kosong.
- `isEmpty(const Queue* q)`

  Mengecek apakah queue kosong. Jika front == nullptr, berarti tidak ada data di dalam queue.
- `isFull(const Queue* q)`

  Mengecek apakah queue penuh (circular array).
  - Jika kosong → pasti belum penuh.
  - nextRearIdx menghitung posisi berikutnya dari rear.
  - Jika posisi berikutnya sama dengan front, berarti queue sudah penuh.
- `Enqueue (Queue* q, int value)`

  Menambahkan elemen ke dalam queue.
  - Jika penuh → error.
  - Jika kosong → front dan rear sama-sama di awal array.
  - Jika tidak → rear maju ke posisi berikutnya (circular).
  - Nilai disimpan di posisi rear.
- `dequeue(Queue* q)`

Menghapus elemen dari depan queue.

- Jika kosong → error.
- Jika hanya 1 elemen → reset jadi kosong (nullptr).
- Jika lebih → front maju ke posisi berikutnya.

- `front(Queue* q)`
  - Mengambil nilai paling depan tanpa menghapus.

    Jika kosong → error.
- `back(Queue* q)`
  - Mengambil nilai paling belakang tanpa menghapus.

    Jika kosong → error.
- `hasilQueue(int n, int k, int inputData[])`

  Fungsi utama untuk memproses data:

  Melakukan iterasi sebanyak n-k+1 (sliding window).

  Setiap iterasi:
  - Queue di-reset.
  - Masukkan k elemen berturut-turut ke queue.
  - Hitung jumlah semua elemen dengan dequeue satu per satu.

  Hasil tiap window dicetak.

queue.h:

- `#ifndef QUEUE_H`

  Mengecek apakah file header ini belum pernah dimasukkan.
- `#define QUEUE_H`

  Menandai bahwa file ini sudah digunakan agar tidak di-include lagi.
- `#define MAX 100`

  Menentukan kapasitas maksimal queue adalah 100 elemen.
- `struct Queue {`

  Membuat struktur data bernama Queue.
- `int data[MAX];`

  Array untuk menyimpan elemen queue.
- `int* front;`

  Pointer yang menunjuk ke elemen paling depan.

- `int* rear;`

  Pointer yang menunjuk ke elemen paling belakang.
- `void init(Queue* q);`

  Deklarasi fungsi untuk menginisialisasi queue.
- `bool isEmpty(const Queue* q);`

  Deklarasi fungsi untuk mengecek apakah queue kosong.
- `bool isFull(const Queue* q);`

  Deklarasi fungsi untuk mengecek apakah queue penuh.
- `void enqueue(Queue* q, int value);`

  Deklarasi fungsi untuk menambahkan elemen ke queue.
- `void dequeue(Queue* q);`

  Deklarasi fungsi untuk menghapus elemen dari queue.
- `int front(const Queue* q);`

  Deklarasi fungsi untuk mengambil nilai elemen depan.
- `int back(const Queue* q);`

  Deklarasi fungsi untuk mengambil nilai elemen belakang.
- `void hasilQueue(int n, int k, int inputData[]);`

  Deklarasi fungsi untuk memproses dan menampilkan hasil.
- `#endif`

  Penutup dari header guard.

TAUTAN GIT

https://github.com/berrypro08/PraktikumAlgoritma-Modul2